

APPLIED RESEARCH PROJECT · CAPSTONE PRESENTATION

A Decision-Oriented Conversational Shopping Assistant

Preference Elicitation · Explainable Recommendations · Lifecycle Support

Chenghui Tan

California State University, East Bay · April 2026

Information Retrieval $\neq$ Decision Support



Cognitive Overload

Users must mentally juggle all constraints and trade-offs themselves



Vague, Evolving Needs

Users rarely know exactly what they want when they start



Preferences Get Overwritten

Add a new constraint — your earlier choices are erased



Missing Critical Info

Delivery time, promos, and availability hidden or absent

The Decision Burden Falls on the User

*Existing tools assume shopping is a search problem.
In reality, users face a decision problem —
and no platform is built to help them solve it.*

- Decision fatigue increases with more options
- Users guess, settle, or abandon purchases
- Post-purchase regret when trade-offs aren't clear
- Trust erodes when recommendations have no rationale

→ We built a system that addresses the decision — not just the search.

Existing Platforms — Where They Fall Short

Google

Shopping

Search & filter

Amazon

Rufus

AI chat in Amazon

ChatGPT

Shopping

Natural language AI

Perplexity

AI answer engine

KEY GAPS ACROSS THE LANDSCAPE

✗ Overwrites preferences mid-conversation

✗ Assumes you already know what you want

✗ Missing delivery, promos & availability

✗ Text heavy — low information density

✗ No preference continuity across turns

✗ Multiple steps to reach product page

✗ Separates reasoning from shopping flow

✗ Opaque ranking — no explanation why

✗ Not built for first-time / cold-start users

→ The common thread: no platform treats shopping as a decision problem.

Search-first tools

→ retrieve results, not decisions

AI chatbots

→ converse, but don't structure trade-offs

All surveyed platforms

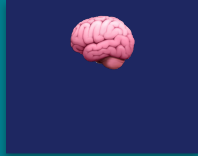
→ no preference continuity across turns

Capability Comparison Across Platforms

Capability	Google Shopping	Amazon Rufus	ChatGPT Shopping	Perplexity	Our System
Preference Elicitation	✗	Partial	Partial	✗	✓
Decision Modeling	✗	✗	✗	✗	✓
Explainable Recommendations	✗	✗	Partial	✓	✓
Delivery / Promo Info	Partial	✗	✗	✗	✓
Preference Continuity	✗	Partial	Partial	✗	✓
Side-by-Side Comparison	✗	✗	✗	✗	✓
Lifecycle Support	✗	✗	✗	✗	✓

→ In our self-evaluation against these platforms, our system addresses all seven gaps. Here is how we built it.

Your Personal AI Shopping Assistant

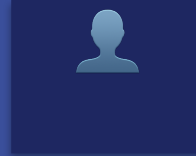


Understand You

Frustration → Category

Decision-Relevant Q&A

ecosystem • placement • privacy • size



Remembers You

Preference Continuity

Visible Diff on Refine

“leaks” → leak-proof • “heavy” → lightweight



Clear Decisions

3 Curated Picks

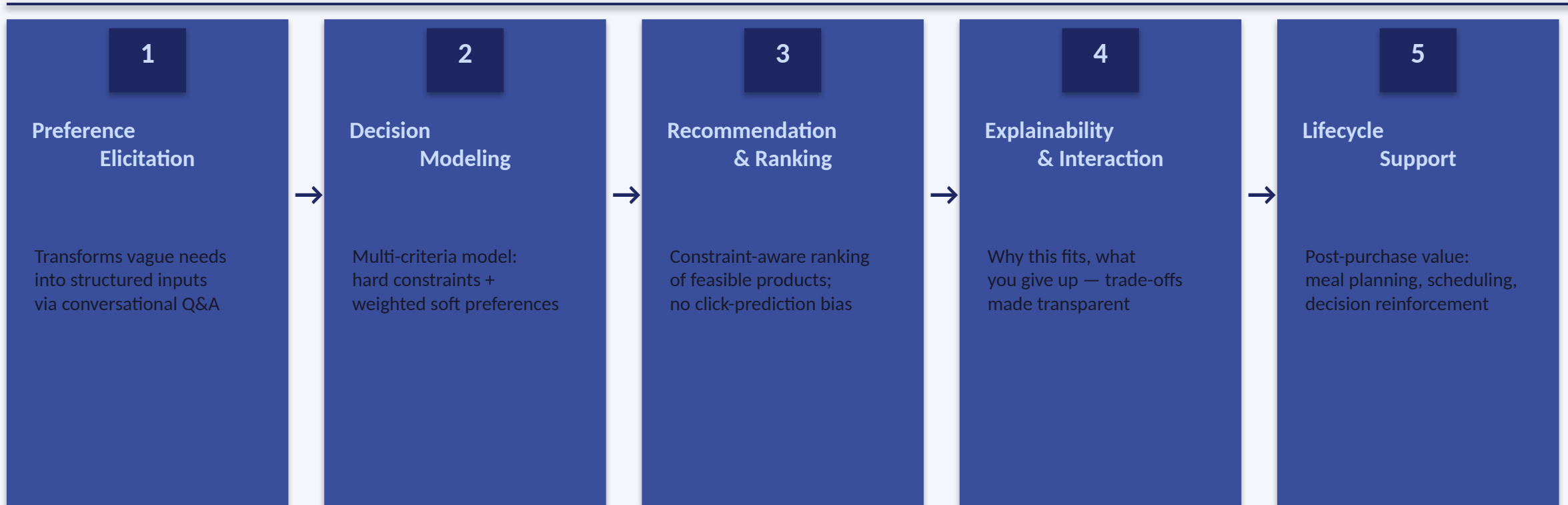
Why-this-fits • Compare

Best Fit • Budget Pick • Stretch Pick



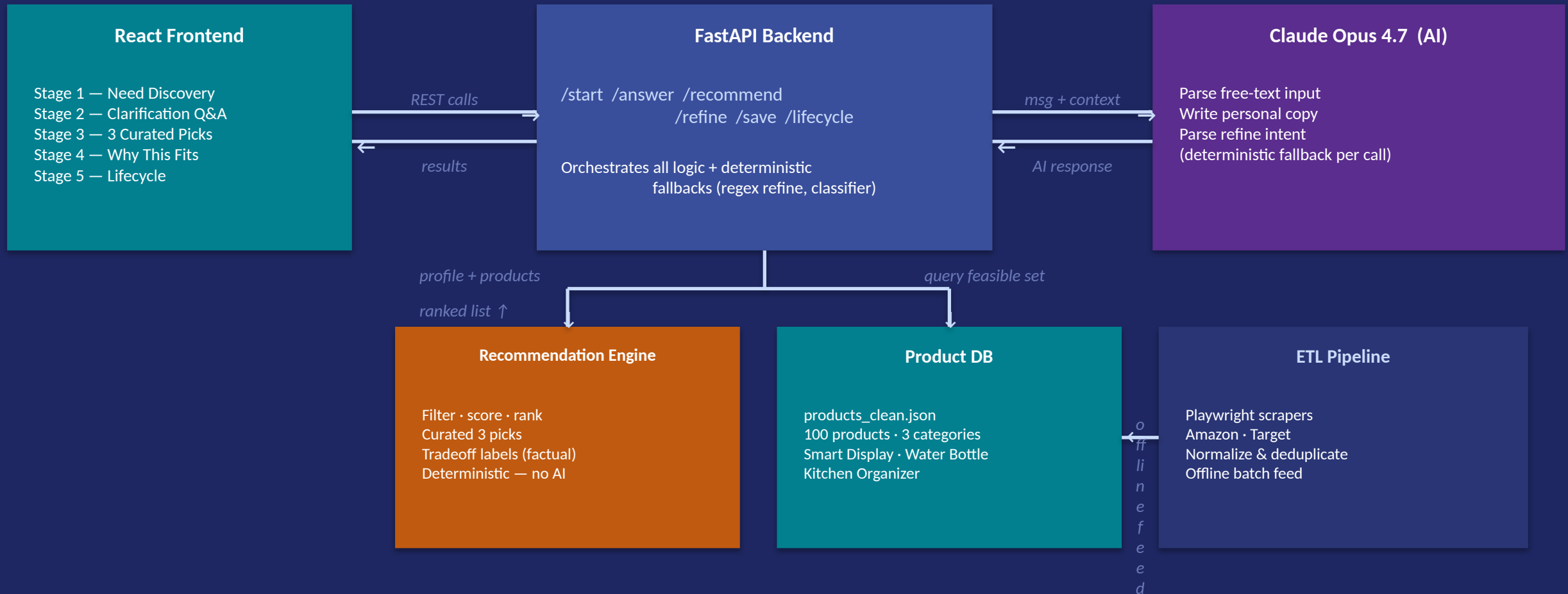
Lifecycle Layer (prototype) — Realistic post-purchase support

Five-Layer Decision Support Framework

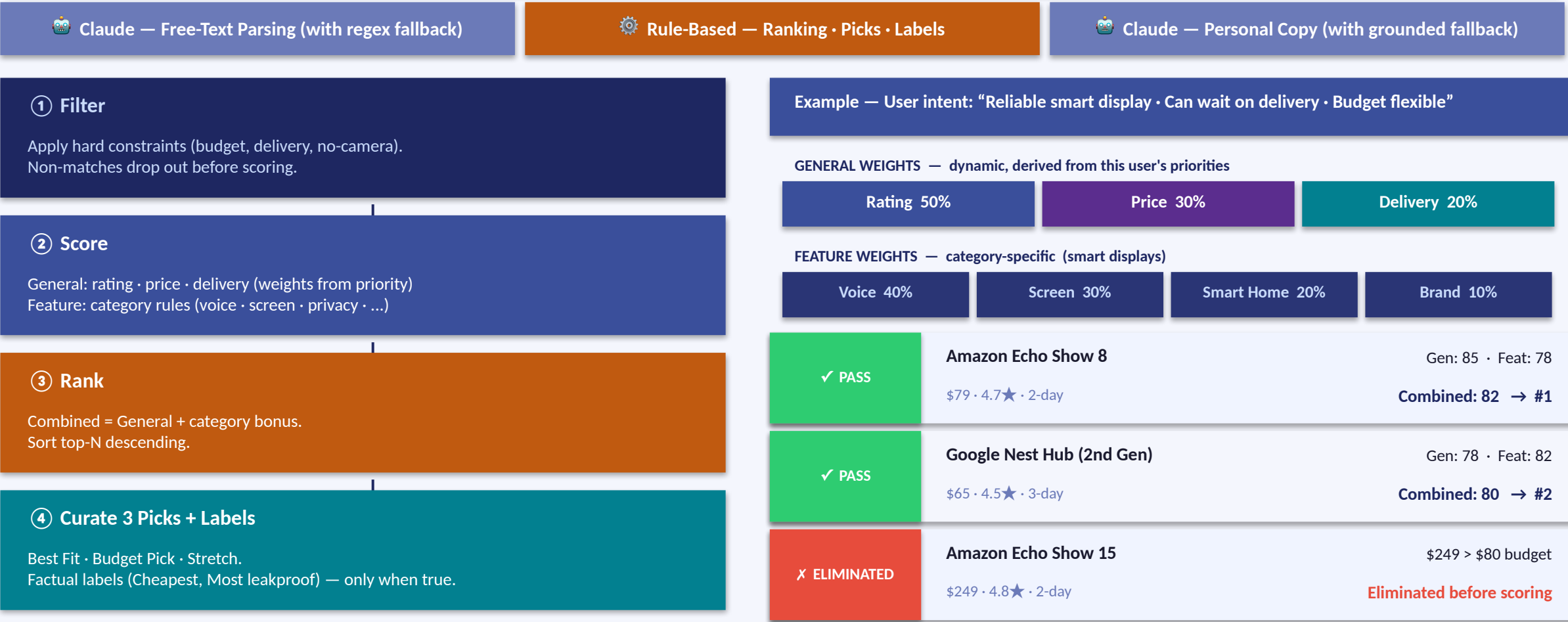


End-to-end: vague need → structured decision → ranked products → explained trade-offs → lifecycle value

Components & How They Communicate



The Deterministic Core



The Supporting Stack — What Feeds the Decision Engine

ETL + Provenance Flags

Playwright scrapers • Amazon • Target
Anti-bot stealth • normalize • dedup
Each field flagged: scraped vs estimated
vs synthesised — no silent fabrication

100 products • 3 categories

Attribute Extraction

screen_inches • ecosystems • has_camera
mounting • capacity_oz • bottle_material
drinking_style • organiser_material
Pulled from titles at load time

feeds ranker, picks, compare view

Claude Opus 4.7 — 3 Call Sites

- ① Parse free-text input
 - ② Write personal product copy
 - ③ Parse refine intent
- Every call has a deterministic fallback

Never on the ranking path

React Frontend — 5 Stages

- ① Need Discovery
- ② Clarification Q&A
- ③ 3 Curated Picks
- ④ Why This Fits (rule-grounded checklist)
- ⑤ Lifecycle (prototype value-extension)

+ Compare overlay • Save-for-later • Diff

Capability Validation & Scenario Walkthrough

Capability Validation

✓ Validation method (multi-agent)

Claude Code ↔ Codex GPT-5.5 cross-review
on the same codebase + family / friends /
classmates user testing

✓ Capability gaps (self-evaluation)

7 of 7 gaps addressed in our walkthroughs;
no surveyed platform addressed more than 2

✓ Decision-critical attributes inline

Screen size, ecosystem, camera, capacity,
material — surfaced on every pick

✓ Trade-offs visible, not hidden

Every pick shows a ✓/✗/? checklist mapping
your chips to match status — no surprises

Demo — Water-Bottle Scenario

① Need Discovery

“Current bottle is heavy and leaks
in my gym bag.” → water_bottle

② Clarification (3 chips)

Gym • Large capacity • (leak inferred
from 'leaks' in the input)

③ 3 Curated Picks

Best Fit: Owala 24oz FreeSip
rules: gym_suitable • large_capacity_pref • leak_proof_match

④ Picks — labels + checklist

Budget Pick: Zak 19oz [Cheapest • Most portable]
Stretch: Owala 32oz [Most capacity • Most leakproof]

⑤ Refine + Compare

'larger' → flips Best Fit to 32oz • diff card shown
Select 3, compare side-by-side with row-winners

Honest Assessment • What This Proves • Next Steps

Limitations

- ⚠ Small dataset — 100 products, 3 categories only
- ⚠ Small-scale user testing — family, friends, classmates only
- ⚠ 72/100 delivery values are estimated (flagged in data)
- ⚠ Rule-based ranking — weights inferred, not learned

Conclusions & Impact

- ✓ Decision support feasible end-to-end without an LLM key
- ✓ Explanations grounded in the same rules the ranker used
- ✓ Visible preference continuity — every refine shows a diff
- ✓ Lifecycle layer is a prototype value-extension concept
- ✓ AI tools disclosed: Claude Opus 4.7 • Codex 5.5 review

Future Work

- Formal user study (SUS, task completion time)
- Live retailer API for real-time pricing
- ML-based ranking from interaction data
- Expand to more product categories
- Mobile-first UI redesign

Thank You

Questions & Discussion

Author:	Chenghui Tan
Institution:	California State University, East Bay (CSUEB)
Project:	Decision-Oriented Conversational Shopping Assistant
Runtime:	React · FastAPI · Claude Opus 4.7 · Python · Playwright (ETL)
Built with:	Claude Code (Opus 4.7) · Codex GPT-5.5 (independent validation)